

Belief Propagation for the Gaussian AR(1) Diffusion Model

A complete derivation and numerical comparison with the precision-matrix score

Prepared for discussion with Jerome

July 11, 2026

Abstract

This note derives belief propagation (BP) for a scalar Gaussian AR(1) clean chain observed through Ornstein-Uhlenbeck (OU) diffusion noise. The goal is to compute the diffusion score

$$s(x, t) = \nabla_x \log p_t(x).$$

There are two equivalent routes. The first route uses Gaussian linear algebra: since the noisy vector is Gaussian, the score is

$$s(x, t) = -\Sigma_t^{-1}x.$$

The second route uses BP on the latent AR(1) chain to compute the posterior marginals

$$p_t(a_i | x_{1:n})$$

and then the posterior means

$$m_i(x, t) = \mathbb{E}[a_i | x_{1:n}].$$

The score is then obtained from

$$s_i(x, t) = -\frac{x_i - \alpha_t m_i(x, t)}{\Delta_t}.$$

The accompanying Python script and notebook implement both routes and verify numerically that they agree up to floating-point precision.

Contents

1	The model	1
1.1	OU noising	2
2	Posterior factorization	3
3	The factor graph	3
4	Generic BP equations	4
5	Forward and backward messages on the chain	5
6	Beliefs and posterior marginals	5
7	Why the posterior mean gives the diffusion score	6

8	Gaussian messages in information form	7
8.1	Multiplying messages	8
8.2	Likelihood in information form	8
9	Gaussian forward recursion	9
10	Gaussian backward recursion	10
11	Final Gaussian beliefs	11
12	The precision-matrix route	12
13	Algebraic equivalence of the two score formulas	12
14	Algorithm summary	13
15	Numerical output from the accompanying script	14
16	Generated plots	15
17	How to use the files	18
18	References and context	18
A	Full Python source code	19

1 The model

We consider a one-dimensional sequence of clean latent variables

$$a_{1:n} = (a_1, \dots, a_n) \in \mathbb{R}^n.$$

The clean sequence follows a stationary Gaussian AR(1) model:

$$a_1 \sim \mathcal{N}(0, 1), \tag{1}$$

$$a_i \mid a_{i-1} \sim \mathcal{N}(\rho a_{i-1}, 1 - \rho^2), \quad i = 2, \dots, n, \tag{2}$$

where

$$|\rho| < 1.$$

It is useful to define

$$q := 1 - \rho^2.$$

Then the transition can also be written as

$$a_i = \rho a_{i-1} + \sqrt{q} \xi_i, \quad \xi_i \sim \mathcal{N}(0, 1). \tag{3}$$

The choice

$$q = 1 - \rho^2$$

keeps the marginal variance equal to one. Indeed, if $\text{Var}(a_{i-1}) = 1$, then

$$\text{Var}(a_i) = \text{Var}(\rho a_{i-1} + \sqrt{q} \xi_i) \quad (4)$$

$$= \rho^2 \text{Var}(a_{i-1}) + q \text{Var}(\xi_i) \quad (5)$$

$$= \rho^2 + q \quad (6)$$

$$= \rho^2 + (1 - \rho^2) \quad (7)$$

$$= 1. \quad (8)$$

Therefore the chain is stationary with

$$a_i \sim \mathcal{N}(0, 1)$$

for every i . The covariance is

$$\text{Cov}(a_i, a_j) = \rho^{|i-j|}. \quad (9)$$

Thus the clean covariance matrix is

$$\Sigma_0(i, j) = \rho^{|i-j|}, \quad 1 \leq i, j \leq n. \quad (10)$$

1.1 OU noising

At diffusion time $t > 0$, we observe the noisy vector

$$x_i = \alpha_t a_i + \sqrt{\Delta_t} z_i, \quad z_i \sim \mathcal{N}(0, 1), \quad (11)$$

independently over i , with

$$\alpha_t = e^{-t}, \quad \Delta_t = 1 - e^{-2t} = 1 - \alpha_t^2. \quad (12)$$

This is the standard OU corruption used in the diffusion setup. The likelihood factor for coordinate i is

$$p_t(x_i | a_i) = \mathcal{N}(x_i; \alpha_t a_i, \Delta_t). \quad (13)$$

Since the noise variables z_i are independent, the full likelihood factorizes as

$$p_t(x_{1:n} | a_{1:n}) = \prod_{i=1}^n \mathcal{N}(x_i; \alpha_t a_i, \Delta_t). \quad (14)$$

2 Posterior factorization

The posterior distribution of the clean variables given the noisy observation is

$$p_t(a_{1:n} | x_{1:n}) = \frac{p_0(a_{1:n}) p_t(x_{1:n} | a_{1:n})}{p_t(x_{1:n})}. \quad (15)$$

Since the denominator does not depend on $a_{1:n}$, we can write

$$p_t(a_{1:n} | x_{1:n}) \propto p_0(a_{1:n}) p_t(x_{1:n} | a_{1:n}). \quad (16)$$

The AR(1) prior factorizes as

$$p_0(a_{1:n}) = p(a_1) \prod_{i=2}^n p(a_i | a_{i-1}). \quad (17)$$

Therefore

$$p_t(a_{1:n} | x_{1:n}) \propto p(a_1) \prod_{i=2}^n p(a_i | a_{i-1}) \prod_{i=1}^n p_t(x_i | a_i). \quad (18)$$

Define the factors

$$\phi_1(a_1) := p(a_1) = \mathcal{N}(a_1; 0, 1), \quad (19)$$

$$\psi_i(a_{i-1}, a_i) := p(a_i | a_{i-1}) = \mathcal{N}(a_i; \rho a_{i-1}, q), \quad i = 2, \dots, n, \quad (20)$$

$$\ell_i(a_i; x_i) := p_t(x_i | a_i) = \mathcal{N}(x_i; \alpha_t a_i, \Delta_t), \quad i = 1, \dots, n. \quad (21)$$

Then the posterior becomes

$$p_t(a_{1:n} | x_{1:n}) = \frac{1}{Z_t(x)} \phi_1(a_1) \left[\prod_{i=2}^n \psi_i(a_{i-1}, a_i) \right] \left[\prod_{i=1}^n \ell_i(a_i; x_i) \right], \quad (22)$$

where

$$Z_t(x) = p_t(x_{1:n}) \quad (23)$$

is the evidence, also called the partition function in statistical physics language.

3 The factor graph

The factor graph is bipartite: variables connect to factors, and factors connect to variables. There are no factor-to-factor edges.

Formally, let

$$\mathcal{G} = (\mathcal{V}, \mathcal{F}, \mathcal{E}). \quad (24)$$

The variable nodes are

$$\mathcal{V} = \{a_1, \dots, a_n\}. \quad (25)$$

The factor nodes are

$$\mathcal{F} = \{\phi_1\} \cup \{\ell_1, \dots, \ell_n\} \cup \{\psi_2, \dots, \psi_n\}. \quad (26)$$

Their scopes are

$$\partial\phi_1 = \{a_1\}, \quad (27)$$

$$\partial\ell_i = \{a_i\}, \quad i = 1, \dots, n, \quad (28)$$

$$\partial\psi_i = \{a_{i-1}, a_i\}, \quad i = 2, \dots, n. \quad (29)$$

Equivalently, the edge set is

$$\begin{aligned} \mathcal{E} = & \{(\phi_1, a_1)\} \cup \{(\ell_i, a_i) : i = 1, \dots, n\} \\ & \cup \{(\psi_i, a_{i-1}), (\psi_i, a_i) : i = 2, \dots, n\}. \end{aligned} \quad (30)$$

Thus

$$\ell_i \not\sim \psi_i. \quad (31)$$

This is because ℓ_i and ψ_i are both factors, and the graph is bipartite.

For an interior node a_i , with $2 \leq i \leq n-1$, the neighborhood is

$$\partial a_i = \{\ell_i, \psi_i, \psi_{i+1}\}. \quad (32)$$

At the left boundary,

$$\partial a_1 = \{\phi_1, \ell_1, \psi_2\}, \quad (33)$$

and at the right boundary,

$$\partial a_n = \{\ell_n, \psi_n\}. \quad (34)$$

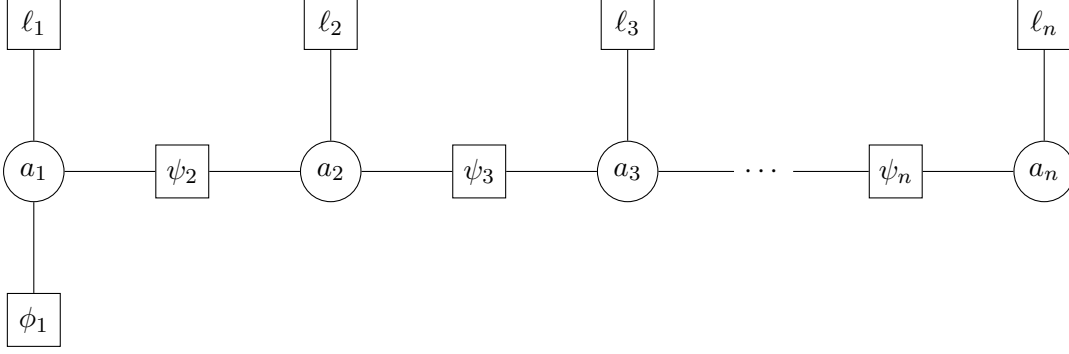


Figure 1: Correct factor graph for the AR(1) posterior. Each likelihood factor ℓ_i attaches only to variable a_i . Each transition factor ψ_i attaches to a_{i-1} and a_i .

4 Generic BP equations

BP has variable-to-factor messages and factor-to-variable messages.

A variable-to-factor message is

$$m_{a_i \rightarrow f}(a_i). \quad (35)$$

A factor-to-variable message is

$$m_{f \rightarrow a_i}(a_i). \quad (36)$$

The generic BP rule from variables to factors is

$$m_{a_i \rightarrow f}(a_i) \propto \prod_{g \in \partial a_i \setminus \{f\}} m_{g \rightarrow a_i}(a_i). \quad (37)$$

This means: to send a message from a_i to f , multiply all incoming messages to a_i , except the message coming from f itself.

The generic BP rule from factors to variables is

$$m_{f \rightarrow a_i}(a_i) \propto \int f(a_{\partial f}) \prod_{a_j \in \partial f \setminus \{a_i\}} m_{a_j \rightarrow f}(a_j) \prod_{a_j \in \partial f \setminus \{a_i\}} da_j. \quad (38)$$

This means: to send a message from factor f to variable a_i , multiply the factor by all incoming messages from the other variables in that factor, then integrate out those other variables.

For unary factors, there is no variable to integrate out. Therefore

$$m_{\ell_i \rightarrow a_i}(a_i) = \ell_i(a_i; x_i), \quad (39)$$

$$m_{\phi_1 \rightarrow a_1}(a_1) = \phi_1(a_1). \quad (40)$$

5 Forward and backward messages on the chain

Because the factor graph is a chain with unary factors attached, it is convenient to summarize messages by two directional messages.

Define $L_i(a_i)$ as the message arriving at a_i from the left side of the graph. It contains the information from the prior and the observations to the left of site i . Define $R_i(a_i)$ as the message arriving at a_i from the right side of the graph. It contains the information from the observations to the right of site i .

The boundary conditions are

$$L_1(a_1) = \phi_1(a_1), \quad R_n(a_n) = 1. \quad (41)$$

The forward recursion is

$$L_{i+1}(a_{i+1}) \propto \int \psi_{i+1}(a_i, a_{i+1}) L_i(a_i) \ell_i(a_i; x_i) da_i, \quad i = 1, \dots, n-1. \quad (42)$$

The backward recursion is

$$R_{i-1}(a_{i-1}) \propto \int \psi_i(a_{i-1}, a_i) R_i(a_i) \ell_i(a_i; x_i) da_i, \quad i = n, n-1, \dots, 2. \quad (43)$$

These equations are just the generic BP rules specialized to the chain.

6 Beliefs and posterior marginals

Once the left and right messages have been computed, the belief at node a_i is obtained by multiplying all information local to that node:

$$b_i(a_i) \propto L_i(a_i) \ell_i(a_i; x_i) R_i(a_i). \quad (44)$$

This is the central local rule:

belief at a_i = left information $\times$ local observation $\times$ right information.

(45)

To make b_i a probability density, normalize it:

$$b_i(a_i) = \frac{L_i(a_i) \ell_i(a_i; x_i) R_i(a_i)}{\int L_i(u) \ell_i(u; x_i) R_i(u) du}. \quad (46)$$

The integration variable in the denominator is written as u only to avoid confusing it with the outside variable a_i .

On this tree factor graph, BP gives the exact posterior marginal:

$$b_i(a_i) = p_t(a_i | x_{1:n}). \quad (47)$$

The posterior mean is

$$m_i(x, t) := \mathbb{E}[a_i | x_{1:n}] = \int a_i b_i(a_i) da_i. \quad (48)$$

This posterior mean is a Bayesian denoiser: it is the estimate of the clean latent value a_i after using the full noisy sequence $x_{1:n}$.

7 Why the posterior mean gives the diffusion score

The noisy marginal is

$$p_t(x_{1:n}) = \int p_0(a_{1:n}) \prod_{j=1}^n \mathcal{N}(x_j; \alpha_t a_j, \Delta_t) da_{1:n}. \quad (49)$$

The score component is

$$s_i(x, t) := \partial_{x_i} \log p_t(x_{1:n}) = \frac{\partial_{x_i} p_t(x_{1:n})}{p_t(x_{1:n})}. \quad (50)$$

We now compute the numerator. Only the likelihood factor involving x_i depends on x_i . Therefore

$$\partial_{x_i} p_t(x_{1:n}) = \partial_{x_i} \int p_0(a_{1:n}) \prod_{j=1}^n \mathcal{N}(x_j; \alpha_t a_j, \Delta_t) da_{1:n} \quad (51)$$

$$= \int p_0(a_{1:n}) [\partial_{x_i} \mathcal{N}(x_i; \alpha_t a_i, \Delta_t)] \prod_{j \neq i} \mathcal{N}(x_j; \alpha_t a_j, \Delta_t) da_{1:n}. \quad (52)$$

For a one-dimensional Gaussian density as a function of x_i ,

$$\mathcal{N}(x_i; \alpha_t a_i, \Delta_t) = \frac{1}{\sqrt{2\pi\Delta_t}} \exp \left[-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t} \right]. \quad (53)$$

Differentiate it:

$$\partial_{x_i} \mathcal{N}(x_i; \alpha_t a_i, \Delta_t) = \mathcal{N}(x_i; \alpha_t a_i, \Delta_t) \partial_{x_i} \left[-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t} \right] \quad (54)$$

$$= \mathcal{N}(x_i; \alpha_t a_i, \Delta_t) \left[-\frac{1}{2\Delta_t} 2(x_i - \alpha_t a_i) \right] \quad (55)$$

$$= \mathcal{N}(x_i; \alpha_t a_i, \Delta_t) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right]. \quad (56)$$

Substitute this derivative into the numerator:

$$\partial_{x_i} p_t(x_{1:n}) = \int p_0(a_{1:n}) \prod_{j=1}^n \mathcal{N}(x_j; \alpha_t a_j, \Delta_t) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right] da_{1:n}. \quad (57)$$

Divide by $p_t(x_{1:n})$:

$$s_i(x, t) = \frac{\partial_{x_i} p_t(x_{1:n})}{p_t(x_{1:n})} \quad (58)$$

$$= \int \frac{p_0(a_{1:n}) \prod_{j=1}^n \mathcal{N}(x_j; \alpha_t a_j, \Delta_t)}{p_t(x_{1:n})} \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right] da_{1:n}. \quad (59)$$

By Bayes' rule,

$$\frac{p_0(a_{1:n}) \prod_{j=1}^n \mathcal{N}(x_j; \alpha_t a_j, \Delta_t)}{p_t(x_{1:n})} = p_t(a_{1:n} | x_{1:n}). \quad (60)$$

Thus

$$s_i(x, t) = \int p_t(a_{1:n} | x_{1:n}) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right] da_{1:n} \quad (61)$$

$$= -\frac{1}{\Delta_t} \left[x_i \int p_t(a_{1:n} | x_{1:n}) da_{1:n} - \alpha_t \int a_i p_t(a_{1:n} | x_{1:n}) da_{1:n} \right] \quad (62)$$

$$= -\frac{1}{\Delta_t} [x_i - \alpha_t \mathbb{E}[a_i | x_{1:n}]]. \quad (63)$$

Therefore

$$\boxed{s_i(x, t) = -\frac{x_i - \alpha_t m_i(x, t)}{\Delta_t}, \quad m_i(x, t) = \mathbb{E}[a_i | x_{1:n}].} \quad (64)$$

This is the bridge between BP and diffusion: BP computes the posterior denoiser $m_i(x, t)$, and the denoiser gives the diffusion score.

8 Gaussian messages in information form

The previous BP equations are functional equations over continuous variables. In the Gaussian case, every message remains Gaussian-shaped. Therefore each message can be stored using two scalar parameters.

We write a Gaussian-shaped message as

$$m(a) \propto \exp\left(-\frac{1}{2}\lambda a^2 + ha\right). \quad (65)$$

Here

$$\lambda = \text{precision}, \quad h = \text{information parameter}.$$

If the normalized Gaussian has mean μ and variance v , then

$$\lambda = \frac{1}{v}, \quad h = \frac{\mu}{v}. \quad (66)$$

Conversely,

$$v = \frac{1}{\lambda}, \quad \mu = \frac{h}{\lambda}. \quad (67)$$

To see this, expand a Gaussian:

$$\mathcal{N}(a; \mu, v) \propto \exp\left[-\frac{(a - \mu)^2}{2v}\right] \quad (68)$$

$$= \exp\left[-\frac{a^2 - 2\mu a + \mu^2}{2v}\right] \quad (69)$$

$$= \exp\left[-\frac{1}{2v}a^2 + \frac{\mu}{v}a - \frac{\mu^2}{2v}\right] \quad (70)$$

$$\propto \exp\left[-\frac{1}{2}\left(\frac{1}{v}\right)a^2 + \left(\frac{\mu}{v}\right)a\right]. \quad (71)$$

Thus $\lambda = 1/v$ and $h = \mu/v$.

8.1 Multiplying messages

Suppose

$$m_1(a) \propto \exp\left(-\frac{1}{2}\lambda_1 a^2 + h_1 a\right), \quad (72)$$

$$m_2(a) \propto \exp\left(-\frac{1}{2}\lambda_2 a^2 + h_2 a\right). \quad (73)$$

Then

$$m_1(a)m_2(a) \propto \exp\left(-\frac{1}{2}\lambda_1 a^2 + h_1 a\right) \exp\left(-\frac{1}{2}\lambda_2 a^2 + h_2 a\right) \quad (74)$$

$$= \exp\left[-\frac{1}{2}(\lambda_1 + \lambda_2)a^2 + (h_1 + h_2)a\right]. \quad (75)$$

Therefore multiplication corresponds to addition:

$$\lambda_{\text{product}} = \lambda_1 + \lambda_2, \quad h_{\text{product}} = h_1 + h_2. \quad (76)$$

This is why Gaussian BP is algebraically simple.

8.2 Likelihood in information form

The local likelihood is

$$\ell_i(a_i; x_i) = \mathcal{N}(x_i; \alpha_t a_i, \Delta_t). \quad (77)$$

As a function of a_i , ignore constants independent of a_i :

$$\ell_i(a_i; x_i) \propto \exp \left[-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t} \right]. \quad (78)$$

Expand the square:

$$(x_i - \alpha_t a_i)^2 = x_i^2 - 2\alpha_t x_i a_i + \alpha_t^2 a_i^2. \quad (79)$$

Therefore

$$-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t} = -\frac{x_i^2}{2\Delta_t} + \frac{\alpha_t x_i}{\Delta_t} a_i - \frac{\alpha_t^2}{2\Delta_t} a_i^2 \quad (80)$$

$$= \text{constant in } a_i - \frac{1}{2} \left(\frac{\alpha_t^2}{\Delta_t} \right) a_i^2 + \left(\frac{\alpha_t x_i}{\Delta_t} \right) a_i. \quad (81)$$

Thus

$$\ell_i(a_i; x_i) \propto \exp \left[-\frac{1}{2} \lambda_\ell a_i^2 + h_{\ell,i} a_i \right], \quad (82)$$

with

$$\lambda_\ell = \frac{\alpha_t^2}{\Delta_t}, \quad h_{\ell,i} = \frac{\alpha_t x_i}{\Delta_t}. \quad (83)$$

The prior $\phi_1(a_1) = \mathcal{N}(a_1; 0, 1)$ has

$$\lambda_\phi = 1, \quad h_\phi = 0. \quad (84)$$

9 Gaussian forward recursion

Let $L_i(a_i)$ have information parameters (λ_i^L, h_i^L) . The boundary condition is

$$\lambda_1^L = 1, \quad h_1^L = 0. \quad (85)$$

At site i , before sending information to the right, multiply L_i by the local likelihood ℓ_i . By (76), the product message has

$$\lambda_i^{\rightarrow} = \lambda_i^L + \lambda_\ell, \quad h_i^{\rightarrow} = h_i^L + h_{\ell,i}. \quad (86)$$

Thus

$$M_i^{\rightarrow}(a_i) \propto L_i(a_i) \ell_i(a_i; x_i) \quad (87)$$

is Gaussian with

$$\mu_i^{\rightarrow} = \frac{h_i^{\rightarrow}}{\lambda_i^{\rightarrow}}, \quad v_i^{\rightarrow} = \frac{1}{\lambda_i^{\rightarrow}}. \quad (88)$$

The AR(1) transition says

$$a_{i+1} = \rho a_i + \sqrt{q} \xi_i, \quad \xi_i \sim \mathcal{N}(0, 1). \quad (89)$$

If

$$a_i \sim \mathcal{N}(\mu_i^{\rightarrow}, v_i^{\rightarrow}),$$

then

$$\mathbb{E}[a_{i+1}] = \mathbb{E}[\rho a_i + \sqrt{q} \xi_i] = \rho \mathbb{E}[a_i] + \sqrt{q} \mathbb{E}[\xi_i] = \rho \mu_i^{\rightarrow}, \quad (90)$$

$$\text{Var}(a_{i+1}) = \text{Var}(\rho a_i + \sqrt{q} \xi_i) \quad (91)$$

$$= \rho^2 \text{Var}(a_i) + q \text{Var}(\xi_i) \quad (92)$$

$$= \rho^2 v_i^{\rightarrow} + q. \quad (93)$$

Therefore the next left message L_{i+1} has mean and variance

$$\mu_{i+1}^L = \rho \mu_i^{\rightarrow}, \quad v_{i+1}^L = q + \rho^2 v_i^{\rightarrow}. \quad (94)$$

Convert back to information form:

$$\lambda_{i+1}^L = \frac{1}{v_{i+1}^L} = \frac{1}{q + \rho^2 v_i^{\rightarrow}} = \frac{1}{q + \rho^2 / \lambda_i^{\rightarrow}}, \quad (95)$$

$$h_{i+1}^L = \lambda_{i+1}^L \mu_{i+1}^L = \lambda_{i+1}^L \rho \mu_i^{\rightarrow} = \lambda_{i+1}^L \rho \frac{h_i^{\rightarrow}}{\lambda_i^{\rightarrow}}. \quad (96)$$

So the forward recursion is

$$\boxed{\lambda_{i+1}^L = \frac{1}{q + \rho^2 / \lambda_i^{\rightarrow}}, \quad h_{i+1}^L = \lambda_{i+1}^L \rho \frac{h_i^{\rightarrow}}{\lambda_i^{\rightarrow}}.} \quad (97)$$

10 Gaussian backward recursion

Let $R_i(a_i)$ have information parameters (λ_i^R, h_i^R) . The boundary condition is

$$\lambda_n^R = 0, \quad h_n^R = 0. \quad (98)$$

This represents the constant message $R_n(a_n) = 1$, since

$$\exp\left(-\frac{1}{2} \cdot 0 \cdot a_n^2 + 0 \cdot a_n\right) = 1.$$

At site i , multiply R_i by the local likelihood. The product has

$$\lambda_i^{\leftarrow} = \lambda_i^R + \lambda_\ell, \quad h_i^{\leftarrow} = h_i^R + h_{\ell,i}. \quad (99)$$

Thus

$$M_i^{\leftarrow}(a_i) \propto R_i(a_i) \ell_i(a_i; x_i) \quad (100)$$

is Gaussian with

$$\mu_i^{\leftarrow} = \frac{h_i^{\leftarrow}}{\lambda_i^{\leftarrow}}, \quad v_i^{\leftarrow} = \frac{1}{\lambda_i^{\leftarrow}}. \quad (101)$$

Now compute

$$R_{i-1}(a_{i-1}) \propto \int \mathcal{N}(a_i; \rho a_{i-1}, q) M_i^{\leftarrow}(a_i) da_i. \quad (102)$$

Since

$$M_i^{\leftarrow}(a_i) \propto \mathcal{N}(a_i; \mu_i^{\leftarrow}, v_i^{\leftarrow}),$$

we need the integral

$$\int \mathcal{N}(a_i; \rho a_{i-1}, q) \mathcal{N}(a_i; \mu_i^{\leftarrow}, v_i^{\leftarrow}) da_i. \quad (103)$$

The Gaussian product identity gives

$$\int \mathcal{N}(y; m_1, v_1) \mathcal{N}(y; m_2, v_2) dy = \mathcal{N}(m_1; m_2, v_1 + v_2). \quad (104)$$

Here

$$y = a_i, \quad m_1 = \rho a_{i-1}, \quad v_1 = q, \quad m_2 = \mu_i^{\leftarrow}, \quad v_2 = v_i^{\leftarrow}.$$

Therefore

$$R_{i-1}(a_{i-1}) \propto \mathcal{N}(\mu_i^{\leftarrow}; \rho a_{i-1}, q + v_i^{\leftarrow}). \quad (105)$$

As a function of a_{i-1} , this is

$$R_{i-1}(a_{i-1}) \propto \exp \left[-\frac{(\mu_i^{\leftarrow} - \rho a_{i-1})^2}{2(q + v_i^{\leftarrow})} \right] \quad (106)$$

$$= \exp \left[-\frac{(\mu_i^{\leftarrow})^2 - 2\rho\mu_i^{\leftarrow}a_{i-1} + \rho^2 a_{i-1}^2}{2(q + v_i^{\leftarrow})} \right] \quad (107)$$

$$\propto \exp \left[-\frac{1}{2} \left(\frac{\rho^2}{q + v_i^{\leftarrow}} \right) a_{i-1}^2 + \left(\frac{\rho\mu_i^{\leftarrow}}{q + v_i^{\leftarrow}} \right) a_{i-1} \right]. \quad (108)$$

Thus

$$\lambda_{i-1}^R = \frac{\rho^2}{q + v_i^{\leftarrow}}, \quad h_{i-1}^R = \frac{\rho\mu_i^{\leftarrow}}{q + v_i^{\leftarrow}}. \quad (109)$$

Now substitute

$$v_i^{\leftarrow} = \frac{1}{\lambda_i^{\leftarrow}}, \quad \mu_i^{\leftarrow} = \frac{h_i^{\leftarrow}}{\lambda_i^{\leftarrow}}.$$

For the precision:

$$\lambda_{i-1}^R = \frac{\rho^2}{q + 1/\lambda_i^{\leftarrow}} \quad (110)$$

$$= \frac{\rho^2}{(q\lambda_i^{\leftarrow} + 1)/\lambda_i^{\leftarrow}} \quad (111)$$

$$= \frac{\rho^2 \lambda_i^{\leftarrow}}{q\lambda_i^{\leftarrow} + 1}. \quad (112)$$

For the information parameter:

$$h_{i-1}^R = \frac{\rho(h_i^{\leftarrow}/\lambda_i^{\leftarrow})}{q + 1/\lambda_i^{\leftarrow}} \quad (113)$$

$$= \frac{\rho h_i^{\leftarrow}/\lambda_i^{\leftarrow}}{(q\lambda_i^{\leftarrow} + 1)/\lambda_i^{\leftarrow}} \quad (114)$$

$$= \frac{\rho h_i^{\leftarrow}}{q\lambda_i^{\leftarrow} + 1}. \quad (115)$$

So the backward recursion is

$$\boxed{\lambda_{i-1}^R = \frac{\rho^2 \lambda_i^{\leftarrow}}{q\lambda_i^{\leftarrow} + 1}, \quad h_{i-1}^R = \frac{\rho h_i^{\leftarrow}}{q\lambda_i^{\leftarrow} + 1}.} \quad (116)$$

11 Final Gaussian beliefs

The belief is

$$b_i(a_i) \propto L_i(a_i) \ell_i(a_i; x_i) R_i(a_i). \quad (117)$$

Since multiplication adds information parameters,

$$\lambda_i^b = \lambda_i^L + \lambda_\ell + \lambda_i^R, \quad h_i^b = h_i^L + h_{\ell,i} + h_i^R. \quad (118)$$

Hence

$$b_i(a_i) = \mathcal{N}(a_i; \mu_i^b, v_i^b), \quad (119)$$

where

$$v_i^b = \frac{1}{\lambda_i^b}, \quad \mu_i^b = \frac{h_i^b}{\lambda_i^b}. \quad (120)$$

Therefore

$$m_i(x, t) = \mathbb{E}[a_i | x_{1:n}] = \mu_i^b. \quad (121)$$

The BP score is then

$$s_i^{\text{BP}}(x, t) = -\frac{x_i - \alpha_t \mu_i^b}{\Delta_t}. \quad (122)$$

12 The precision-matrix route

Because the model is Gaussian, the noisy vector is also Gaussian. In vector form,

$$x = \alpha_t a + \sqrt{\Delta_t} z, \quad a \sim \mathcal{N}(0, \Sigma_0), \quad z \sim \mathcal{N}(0, I). \quad (123)$$

Therefore

$$\Sigma_t := \text{Cov}(x) = \text{Cov}(\alpha_t a + \sqrt{\Delta_t} z) \quad (124)$$

$$= \alpha_t^2 \text{Cov}(a) + \Delta_t \text{Cov}(z) \quad (125)$$

$$= \alpha_t^2 \Sigma_0 + \Delta_t I. \quad (126)$$

Thus

$$x \sim \mathcal{N}(0, \Sigma_t). \quad (127)$$

The log-density is

$$\log p_t(x) = -\frac{1}{2} x^\top \Sigma_t^{-1} x - \frac{1}{2} \log \det(2\pi \Sigma_t). \quad (128)$$

The second term does not depend on x . Differentiating the quadratic term gives

$$\nabla_x \left(-\frac{1}{2} x^\top \Sigma_t^{-1} x \right) = -\Sigma_t^{-1} x, \quad (129)$$

because Σ_t^{-1} is symmetric. Hence

$$\boxed{s^{\text{matrix}}(x, t) = -\Sigma_t^{-1} x.} \quad (130)$$

The posterior mean under the Gaussian linear model is

$$\mathbb{E}[a | x] = \text{Cov}(a, x) \text{Cov}(x, x)^{-1} x. \quad (131)$$

Now

$$\text{Cov}(a, x) = \text{Cov}(a, \alpha_t a + \sqrt{\Delta_t} z) \quad (132)$$

$$= \alpha_t \text{Cov}(a, a) + \sqrt{\Delta_t} \text{Cov}(a, z) \quad (133)$$

$$= \alpha_t \Sigma_0, \quad (134)$$

since a and z are independent. Also $\text{Cov}(x, x) = \Sigma_t$. Therefore

$$\boxed{\mathbb{E}[a | x] = \alpha_t \Sigma_0 \Sigma_t^{-1} x.} \quad (135)$$

13 Algebraic equivalence of the two score formulas

Start from the BP score formula:

$$s^{\text{BP}}(x, t) = -\frac{x - \alpha_t \mathbb{E}[a | x]}{\Delta_t}. \quad (136)$$

Using the Gaussian posterior mean,

$$\mathbb{E}[a | x] = \alpha_t \Sigma_0 \Sigma_t^{-1} x. \quad (137)$$

Substitute this into the BP score:

$$s^{\text{BP}}(x, t) = -\frac{x - \alpha_t (\alpha_t \Sigma_0 \Sigma_t^{-1} x)}{\Delta_t} \quad (138)$$

$$= -\frac{x - \alpha_t^2 \Sigma_0 \Sigma_t^{-1} x}{\Delta_t}. \quad (139)$$

Now use

$$\Sigma_t = \alpha_t^2 \Sigma_0 + \Delta_t I. \quad (140)$$

Rearrange it:

$$\Delta_t I = \Sigma_t - \alpha_t^2 \Sigma_0. \quad (141)$$

Multiply both sides by $\Sigma_t^{-1} x$:

$$\Delta_t I \Sigma_t^{-1} x = (\Sigma_t - \alpha_t^2 \Sigma_0) \Sigma_t^{-1} x \quad (142)$$

$$\Delta_t \Sigma_t^{-1} x = \Sigma_t \Sigma_t^{-1} x - \alpha_t^2 \Sigma_0 \Sigma_t^{-1} x \quad (143)$$

$$\Delta_t \Sigma_t^{-1} x = x - \alpha_t^2 \Sigma_0 \Sigma_t^{-1} x. \quad (144)$$

Therefore

$$\frac{x - \alpha_t^2 \Sigma_0 \Sigma_t^{-1} x}{\Delta_t} = \Sigma_t^{-1} x. \quad (145)$$

Substitute this into the BP score:

$$s^{\text{BP}}(x, t) = -\Sigma_t^{-1} x = s^{\text{matrix}}(x, t). \quad (146)$$

So the two algorithms must agree:

$$\boxed{s^{\text{BP}}(x, t) = s^{\text{matrix}}(x, t).} \quad (147)$$

14 Algorithm summary

Given $x_{1:n}$, ρ , and $t > 0$:

1. Set

$$\alpha_t = e^{-t}, \quad \Delta_t = 1 - e^{-2t}, \quad q = 1 - \rho^2.$$

2. Set likelihood information parameters

$$\lambda_\ell = \frac{\alpha_t^2}{\Delta_t}, \quad h_{\ell,i} = \frac{\alpha_t x_i}{\Delta_t}.$$

3. Initialize the left messages:

$$\lambda_1^L = 1, \quad h_1^L = 0.$$

4. For $i = 1, \dots, n-1$, compute

$$\lambda_i^{\rightarrow} = \lambda_i^L + \lambda_\ell, \quad h_i^{\rightarrow} = h_i^L + h_{\ell,i},$$

then

$$\lambda_{i+1}^L = \frac{1}{q + \rho^2/\lambda_i^{\rightarrow}}, \quad h_{i+1}^L = \lambda_{i+1}^L \rho \frac{h_i^{\rightarrow}}{\lambda_i^{\rightarrow}}.$$

5. Initialize the right messages:

$$\lambda_n^R = 0, \quad h_n^R = 0.$$

6. For $i = n, n-1, \dots, 2$, compute

$$\lambda_i^{\leftarrow} = \lambda_i^R + \lambda_\ell, \quad h_i^{\leftarrow} = h_i^R + h_{\ell,i},$$

then

$$\lambda_{i-1}^R = \frac{\rho^2 \lambda_i^{\leftarrow}}{q \lambda_i^{\leftarrow} + 1}, \quad h_{i-1}^R = \frac{\rho h_i^{\leftarrow}}{q \lambda_i^{\leftarrow} + 1}.$$

7. Compute the belief parameters:

$$\lambda_i^b = \lambda_i^L + \lambda_\ell + \lambda_i^R, \quad h_i^b = h_i^L + h_{\ell,i} + h_i^R.$$

8. Compute the posterior mean and variance:

$$\mu_i^b = \frac{h_i^b}{\lambda_i^b}, \quad v_i^b = \frac{1}{\lambda_i^b}.$$

9. Compute the BP score:

$$s_i^{\text{BP}}(x, t) = -\frac{x_i - \alpha_t \mu_i^b}{\Delta_t}.$$

10. Compute the matrix score for comparison:

$$\Sigma_0(i, j) = \rho^{|i-j|}, \quad \Sigma_t = \alpha_t^2 \Sigma_0 + \Delta_t I,$$

$$s^{\text{matrix}}(x, t) = -\Sigma_t^{-1} x.$$

15 Numerical output from the accompanying script

The Python script creates one synthetic AR(1) sample, corrupts it with OU noise, computes the score using both routes, and compares the results. With the default parameters

$$n = 40, \quad \rho = 0.7, \quad t = 0.8,$$

the script writes the following summary.

```
Gaussian AR(1) diffusion BP check
=====
n                = 40
rho              = 0.70000000
t                = 0.80000000
alpha = exp(-t)  = 0.449328964117
Delta = 1-alpha^2 = 0.798103482005

Numerical agreement
-----
max |score_BP - score_matrix|      = 6.6613381477509392e-16
max |mean_BP  - mean_matrix|       = 4.8572257327350599e-16
max |var_BP   - var_matrix|        = 2.2204460492503131e-16

First five entries
-----
i      a_true      x      mean_BP      score_BP
1      -0.98912135  -0.71300832  -0.61114443  0.54930649
2      -0.95503715  -0.94590627  -0.69323797  0.79490240
3       0.25123660  -0.87925506  -0.60423629  0.76149799
4       0.31439106  -0.91779936  -0.34363189  0.95651206
5       0.87725006   1.11913112   0.18592295  -1.29756426
```

The differences are at the level of floating-point roundoff. This is exactly what we expect from the algebraic equivalence shown above.

16 Generated plots

The first plot shows the clean latent sample, the noisy observation, and the posterior mean computed by both methods.

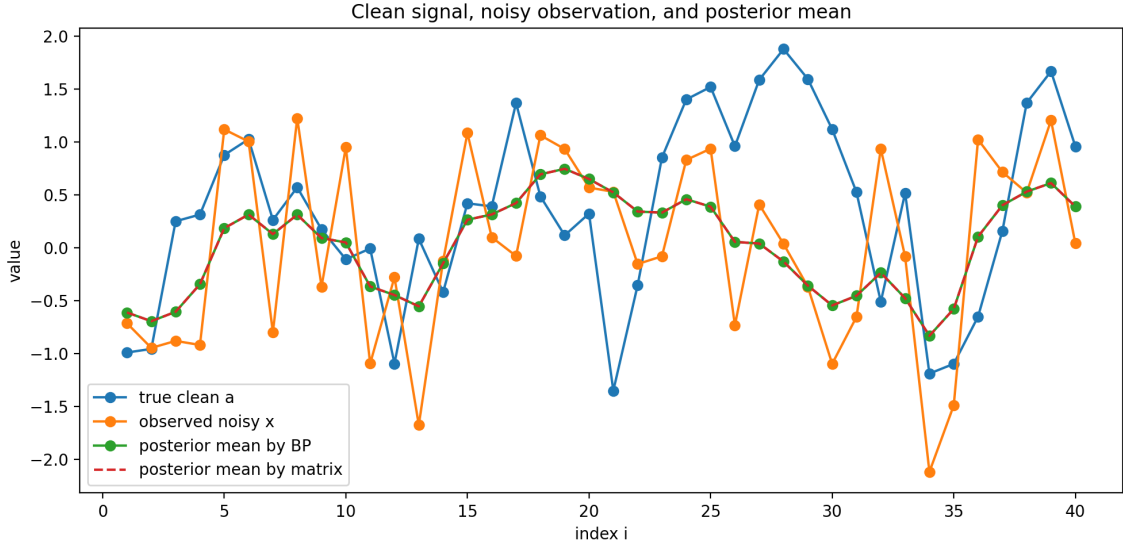


Figure 2: The posterior mean is the Bayesian denoiser $\mathbb{E}[a_i | x_{1:n}]$. The BP and matrix posterior means overlap.

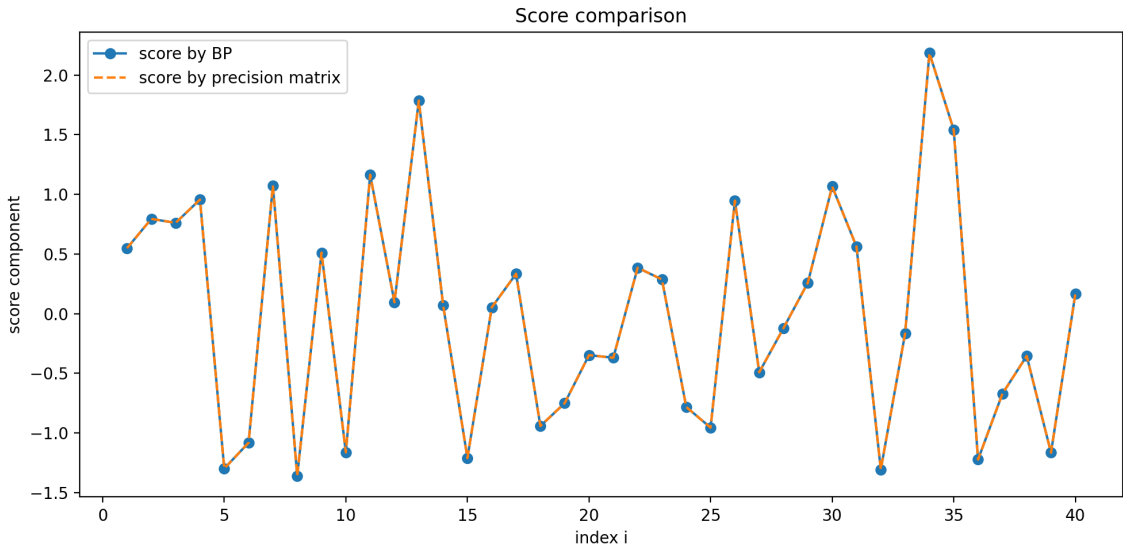


Figure 3: Score components computed by BP and by the precision matrix. The curves overlap.

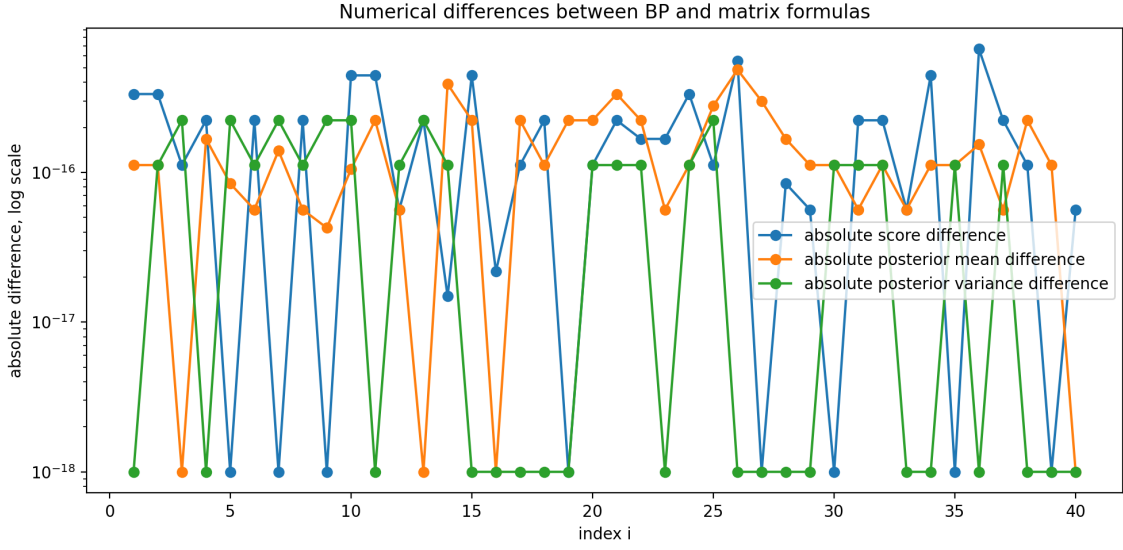


Figure 4: Absolute differences between BP and matrix outputs. The scale is logarithmic.

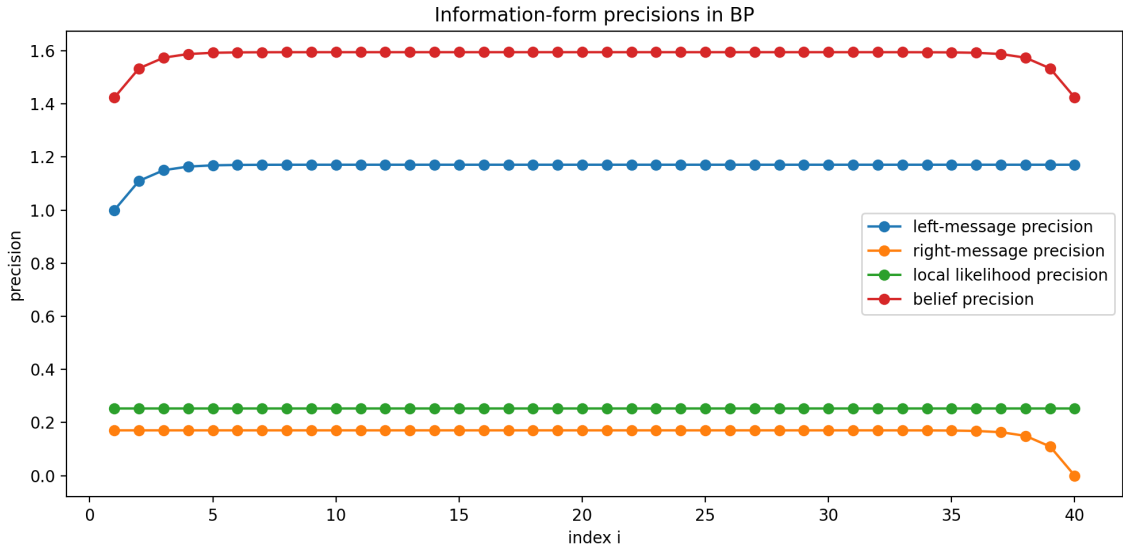


Figure 5: Information-form precisions appearing in BP. The belief precision is the sum of left-message, local-likelihood, and right-message precisions.

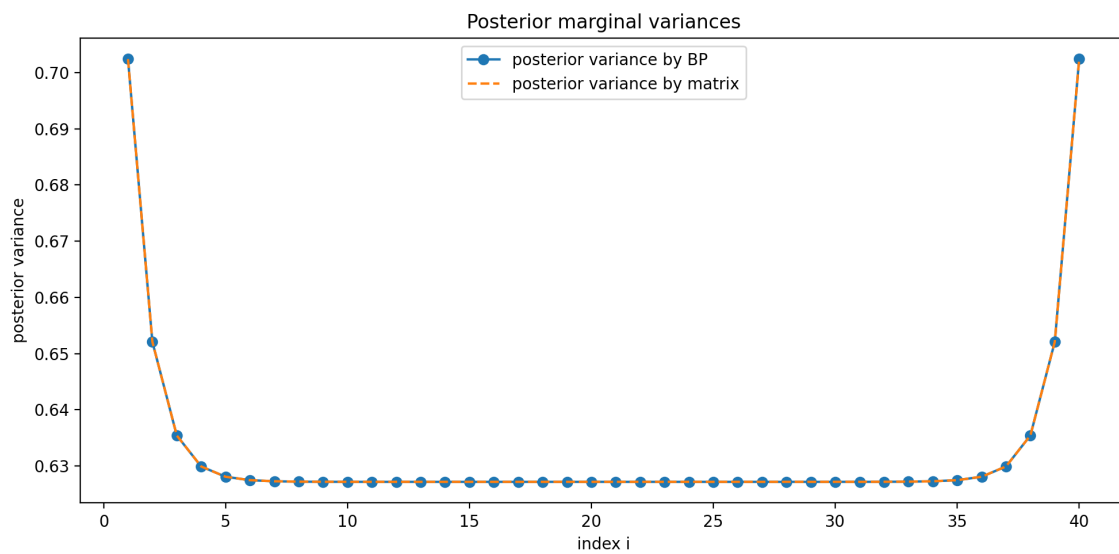


Figure 6: Posterior marginal variances from BP and from the Gaussian matrix formula.

17 How to use the files

The project contains four main files:

- `bp_gaussian_ar1_diffusion.tex`: the LaTeX source for this document.
- `bp_gaussian_ar1_diffusion.pdf`: the compiled PDF.
- `bp_gaussian_ar1_diffusion.py`: a standalone Python script.
- `bp_gaussian_ar1_diffusion.ipynb`: a notebook version for interactive experiments and plots.

To run the Python script from a terminal:

```
python bp_gaussian_ar1_diffusion.py --n 40 --rho 0.7 --t 0.8 --seed 123
```

This creates a folder called `figures_bp_ar1` containing the plots and a text summary. You can modify n , ρ , t , and the random seed from the command line.

18 References and context

The OU noising model and the identity relating the score to a posterior expectation are standard in the diffusion-model literature. In the notation used here, the identity is

$$\nabla_{x_i} \log p_t(x) = -\frac{x_i - \alpha_t \mathbb{E}[a_i | x]}{\Delta_t}.$$

The BP equations used here are the usual sum-product equations on factor graphs. In continuous variables, messages are densities; in the Gaussian case, these densities close exactly inside the Gaussian family and can be represented by precision and information parameters.

References

- [1] G. Biroli and M. Mezard. Generative diffusion in very large dimensions. arXiv:2306.03518, 2023.
- [2] M. Mezard. Generative diffusion updated notes. Lecture notes, December 2025.
- [3] M. Mezard and A. Montanari. Information, Physics, and Computation. Oxford University Press, 2009; draft chapters circulated in 2008.
- [4] G. Genovese and A. Piana. Derivation of the AMP equations from belief propagation for the ℓ_2 minimisation problem. arXiv:2602.15191, 2026.

A Full Python source code

```
"""
Belief propagation for the Gaussian AR(1) diffusion model.

This script compares two mathematically equivalent ways of computing the
score of the noisy distribution  $p_t(x)$ :

1. Matrix route:
     $x \sim N(0, \Sigma_t)$ , so  $\text{score}(x, t) = -\Sigma_t^{-1} x$ .

2. BP route:
    compute the posterior marginals  $p(a_i | x_1, \dots, x_n)$  by Gaussian
    belief propagation on the AR(1) chain, then use
     $\text{score}_i(x, t) = -(x_i - \alpha_t E[a_i | x]) / \Delta_t$ .

Running this file creates numerical checks and figures in the chosen output
folder.
"""

from __future__ import annotations

import argparse
from pathlib import Path
from typing import Dict, Tuple

import numpy as np
import matplotlib.pyplot as plt

Array = np.ndarray

def diffusion_params(t: float) -> Tuple[float, float]:
    """Return  $\alpha_t = \exp(-t)$  and  $\Delta_t = 1 - \exp(-2t)$ ."""
    if t <= 0:
        raise ValueError("This implementation expects  $t > 0$  because  $\Delta_t$  must be positive.")
    alpha = float(np.exp(-t))
    Delta = float(1.0 - alpha**2)
    if Delta <= 0:
        raise ValueError("Delta_t is not positive. Use  $t > 0$ .")
    return alpha, Delta

def ar1_covariance(n: int, rho: float) -> Array:
    """Clean stationary AR(1) covariance:  $\Sigma_0[i, j] = \rho^{|i-j|}$ ."""
    if n <= 0:
        raise ValueError("n must be positive.")
    if abs(rho) >= 1:
        raise ValueError("Stationary AR(1) requires  $|\rho| < 1$ .")
    idx = np.arange(n)
    return rho ** np.abs(idx[:, None] - idx[None, :])

def sample_clean_ar1(n: int, rho: float, seed: int = 0) -> Array:
    """Sample a clean stationary AR(1) chain with marginal variance one."""
    if abs(rho) >= 1:
        raise ValueError("Stationary AR(1) requires  $|\rho| < 1$ .")
    rng = np.random.default_rng(seed)
    q = 1.0 - rho**2
    a = np.empty(n)
    a[0] = rng.normal()
    for i in range(1, n):
        a[i] = rho * a[i - 1] + np.sqrt(q) * rng.normal()
    return a
```

```

def sample_noisy_ar1(n: int, rho: float, t: float, seed: int = 0) -> Tuple[Array, Array]:
    """Sample clean a and noisy x from  $x = \alpha a + \sqrt{\Delta} z$ ."""
    alpha, Delta = diffusion_params(t)
    rng = np.random.default_rng(seed + 1)
    a = sample_clean_ar1(n=n, rho=rho, seed=seed)
    x = alpha * a + np.sqrt(Delta) * rng.normal(size=n)
    return a, x

def score_by_precision_matrix(x: Array, rho: float, t: float) -> Dict[str, Array]:
    """
    Direct Gaussian route.

    Clean prior:
         $a \sim N(0, \Sigma_0)$ 

    Forward OU noising:
         $x = \alpha a + \sqrt{\Delta} z$ ,  $z \sim N(0, I)$ 

    Hence:
         $x \sim N(0, \Sigma_t)$ ,  $\Sigma_t = \alpha^2 \Sigma_0 + \Delta I$ .

    Score:
         $\text{score}(x, t) = -\Sigma_t^{-1} x$ .

    Posterior mean:
         $E[a | x] = \alpha \Sigma_0 \Sigma_t^{-1} x$ .

    Posterior covariance:
         $\text{Cov}(a | x) = \Sigma_0 - \alpha^2 \Sigma_0 \Sigma_t^{-1} \Sigma_0$ .
    """
    x = np.asarray(x, dtype=float)
    n = len(x)
    alpha, Delta = diffusion_params(t)

    Sigma0 = ar1_covariance(n, rho)
    Sigma_t = alpha**2 * Sigma0 + Delta * np.eye(n)

    precision_times_x = np.linalg.solve(Sigma_t, x)
    score = -precision_times_x

    posterior_mean = alpha * Sigma0 @ precision_times_x

    # Solve Sigma_t Y = Sigma0 to avoid explicitly inverting Sigma_t.
    Sigma_t_inv_Sigma0 = np.linalg.solve(Sigma_t, Sigma0)
    posterior_cov = Sigma0 - alpha**2 * Sigma0 @ Sigma_t_inv_Sigma0
    posterior_cov = 0.5 * (posterior_cov + posterior_cov.T) # numerical symmetrization
    posterior_var = np.diag(posterior_cov)

    return {
        "score": score,
        "posterior_mean": posterior_mean,
        "posterior_var": posterior_var,
        "posterior_cov": posterior_cov,
        "Sigma0": Sigma0,
        "Sigma_t": Sigma_t,
    }

def score_by_gaussian_bp(x: Array, rho: float, t: float) -> Dict[str, Array]:
    """
    Gaussian BP route on the AR(1) chain.

    Messages are represented in information form:
         $m(a) \propto \exp(-0.5 \lambda a^2 + h a)$ ,
    where  $\lambda$  is the precision and  $h$  is precision times mean.

    The posterior marginal at node  $i$  is
    """

```

```

    b_i(a_i) propto L_i(a_i) ell_i(a_i; x_i) R_i(a_i).

Its mean gives the denoiser  $E[a_i | x]$ , and the score is
score_i = -(x_i - alpha  $E[a_i | x]$ ) / Delta.
"""
x = np.asarray(x, dtype=float)
n = len(x)
if abs(rho) >= 1:
    raise ValueError("Stationary AR(1) requires |rho| < 1.")

alpha, Delta = diffusion_params(t)
q = 1.0 - rho**2

# Likelihood  $ell_i(a_i; x_i) = N(x_i; \alpha a_i, \Delta)$ , seen as a function of  $a_i$ .
# Information-form parameters are:
#    $\lambda_L = \alpha^2 / \Delta$ ,
#    $h_L = \alpha x_i / \Delta$ .
lambda_L = alpha**2 / Delta
h_L = alpha * x / Delta

# Forward messages  $L_i$ .
lambda_L = np.zeros(n)
h_L = np.zeros(n)
lambda_L[0] = 1.0 # prior  $N(0, 1)$ 
h_L[0] = 0.0

for i in range(n - 1):
    #  $M_i^{right}(a_i)$  propto  $L_i(a_i) ell_i(a_i; x_i)$ .
    lambda_msg = lambda_L[i] + lambda_L
    h_msg = h_L[i] + h_L[i]

    # If  $a_i$  has mean  $h_{msg}/\lambda_{msg}$  and variance  $1/\lambda_{msg}$ ,
    # then  $a_{i+1} = \rho a_i + \sqrt{q}$  noise has mean and variance:
    #    $\rho h_{msg}/\lambda_{msg}$ ,
    #    $q + \rho^2/\lambda_{msg}$ .
    lambda_L[i + 1] = 1.0 / (q + rho**2 / lambda_msg)
    h_L[i + 1] = lambda_L[i + 1] * rho * h_msg / lambda_msg

# Backward messages  $R_i$ .
lambda_R = np.zeros(n)
h_R = np.zeros(n)
lambda_R[-1] = 0.0 # constant message  $R_n(a_n) = 1$ 
h_R[-1] = 0.0

for i in range(n - 1, 0, -1):
    #  $M_i^{left}(a_i)$  propto  $R_i(a_i) ell_i(a_i; x_i)$ .
    lambda_msg = lambda_R[i] + lambda_L
    h_msg = h_R[i] + h_L[i]

    # Integrate  $N(a_i; \rho a_{i-1}, q) M_i^{left}(a_i) da_i$ .
    # The result is Gaussian-shaped in  $a_{i-1}$ , with:
    denom = q * lambda_msg + 1.0
    lambda_R[i - 1] = rho**2 * lambda_msg / denom
    h_R[i - 1] = rho * h_msg / denom

# Beliefs  $b_i(a_i)$  propto  $L_i(a_i) ell_i(a_i; x_i) R_i(a_i)$ .
lambda_b = lambda_L + lambda_L + lambda_R
h_b = h_L + h_L + h_R

posterior_mean = h_b / lambda_b
posterior_var = 1.0 / lambda_b
score = -(x - alpha * posterior_mean) / Delta

return {
    "score": score,
    "posterior_mean": posterior_mean,
    "posterior_var": posterior_var,
    "lambda_L": lambda_L,

```

```

        "h_L": h_L,
        "lambda_R": lambda_R,
        "h_R": h_R,
        "lambda_b": lambda_b,
        "h_b": h_b,
        "lambda_l": np.full(n, lambda_l),
        "h_l": h_l,
    }

def compare_methods(x: Array, rho: float, t: float) -> Dict[str, float | Array | Dict[str, Array]]:
    """Run both methods and return outputs plus numerical discrepancies."""
    bp = score_by_gaussian_bp(x, rho, t)
    mat = score_by_precision_matrix(x, rho, t)

    score_err = np.abs(bp["score"] - mat["score"])
    mean_err = np.abs(bp["posterior_mean"] - mat["posterior_mean"])
    var_err = np.abs(bp["posterior_var"] - mat["posterior_var"])

    return {
        "bp": bp,
        "matrix": mat,
        "score_err": score_err,
        "mean_err": mean_err,
        "var_err": var_err,
        "max_score_error": float(np.max(score_err)),
        "max_mean_error": float(np.max(mean_err)),
        "max_var_error": float(np.max(var_err)),
    }

def save_plots(a_true: Array, x: Array, comparison: Dict[str, object], outdir: Path) -> None:
    """Create plots comparing BP and matrix outputs."""
    outdir.mkdir(parents=True, exist_ok=True)
    idx = np.arange(1, len(x) + 1)
    bp = comparison["bp"]
    mat = comparison["matrix"]

    plt.figure(figsize=(10, 5))
    plt.plot(idx, a_true, marker="o", label="true clean a")
    plt.plot(idx, x, marker="o", label="observed noisy x")
    plt.plot(idx, bp["posterior_mean"], marker="o", label="posterior mean by BP")
    plt.plot(idx, mat["posterior_mean"], linestyle="--", label="posterior mean by matrix")
    plt.xlabel("index i")
    plt.ylabel("value")
    plt.title("Clean signal, noisy observation, and posterior mean")
    plt.legend()
    plt.tight_layout()
    plt.savefig(outdir / "clean_noisy_posterior.png", dpi=200)
    plt.close()

    plt.figure(figsize=(10, 5))
    plt.plot(idx, bp["score"], marker="o", label="score by BP")
    plt.plot(idx, mat["score"], linestyle="--", label="score by precision matrix")
    plt.xlabel("index i")
    plt.ylabel("score component")
    plt.title("Score comparison")
    plt.legend()
    plt.tight_layout()
    plt.savefig(outdir / "score_comparison.png", dpi=200)
    plt.close()

    plt.figure(figsize=(10, 5))
    plt.semilogy(idx, comparison["score_err"] + 1e-18, marker="o", label="absolute score difference")
    plt.semilogy(idx, comparison["mean_err"] + 1e-18, marker="o", label="absolute posterior mean difference")
    plt.semilogy(idx, comparison["var_err"] + 1e-18, marker="o", label="absolute posterior variance difference")
    plt.xlabel("index i")

```

```

plt.ylabel("absolute difference, log scale")
plt.title("Numerical differences between BP and matrix formulas")
plt.legend()
plt.tight_layout()
plt.savefig(outdir / "absolute_differences.png", dpi=200)
plt.close()

plt.figure(figsize=(10, 5))
plt.plot(idx, bp["lambda_L"], marker="o", label="left-message precision")
plt.plot(idx, bp["lambda_R"], marker="o", label="right-message precision")
plt.plot(idx, bp["lambda_l"], marker="o", label="local likelihood precision")
plt.plot(idx, bp["lambda_b"], marker="o", label="belief precision")
plt.xlabel("index i")
plt.ylabel("precision")
plt.title("Information-form precisions in BP")
plt.legend()
plt.tight_layout()
plt.savefig(outdir / "bp_message_precisions.png", dpi=200)
plt.close()

plt.figure(figsize=(10, 5))
plt.plot(idx, bp["posterior_var"], marker="o", label="posterior variance by BP")
plt.plot(idx, mat["posterior_var"], linestyle="--", label="posterior variance by matrix")
plt.xlabel("index i")
plt.ylabel("posterior variance")
plt.title("Posterior marginal variances")
plt.legend()
plt.tight_layout()
plt.savefig(outdir / "posterior_variance.png", dpi=200)
plt.close()

def write_summary(
    a_true: Array,
    x: Array,
    rho: float,
    t: float,
    comparison: Dict[str, object],
    outdir: Path,
) -> None:
    """Write a plain-text numerical summary."""
    alpha, Delta = diffusion_params(t)
    lines = []
    lines.append("Gaussian AR(1) diffusion BP check")
    lines.append("=====")
    lines.append(f"n           = {len(x)}")
    lines.append(f"rho          = {rho:.8f}")
    lines.append(f"t            = {t:.8f}")
    lines.append(f"alpha = exp(-t) = {alpha:.12f}")
    lines.append(f"Delta = 1-alpha^2 = {Delta:.12f}")
    lines.append("")
    lines.append("Numerical agreement")
    lines.append("-----")
    lines.append(f"max |score_BP - score_matrix| = {comparison['max_score_error']:.16e}")
    lines.append(f"max |mean_BP - mean_matrix|   = {comparison['max_mean_error']:.16e}")
    lines.append(f"max |var_BP - var_matrix|     = {comparison['max_var_error']:.16e}")
    lines.append("")
    lines.append("First five entries")
    lines.append("-----")
    lines.append(f"i      a_true      x      mean_BP      score_BP")
    bp = comparison["bp"]
    for i in range(min(5, len(x))):
        lines.append(
            f"{i+1:<6d} {a_true[i]:.8f} {x[i]:.8f} "
            f"{bp['posterior_mean'][i]:.8f} {bp['score'][i]:.8f}"
        )
    text = "\n".join(lines) + "\n"
    (outdir / "results_summary.txt").write_text(text, encoding="utf-8")

```



```

print(text)

def run_experiment(n: int, rho: float, t: float, seed: int, outdir: Path) -> Dict[str, object]:
    """Run a complete experiment and save plots plus summary."""
    a_true, x = sample_noisy_ar1(n=n, rho=rho, t=t, seed=seed)
    comparison = compare_methods(x=x, rho=rho, t=t)
    save_plots(a_true=a_true, x=x, comparison=comparison, outdir=outdir)
    write_summary(a_true=a_true, x=x, rho=rho, t=t, comparison=comparison, outdir=outdir)
    return {"a_true": a_true, "x": x, "comparison": comparison}

def main() -> None:
    parser = argparse.ArgumentParser(description="Compare Gaussian BP and precision-matrix scores.")
    parser.add_argument("--n", type=int, default=40, help="sequence length")
    parser.add_argument("--rho", type=float, default=0.7, help="AR(1) correlation parameter")
    parser.add_argument("--t", type=float, default=0.8, help="diffusion time, must be positive")
    parser.add_argument("--seed", type=int, default=123, help="random seed")
    parser.add_argument(
        "--outdir",
        type=Path,
        default=Path("figures_bp_ar1"),
        help="folder where figures and summary are written",
    )
    args = parser.parse_args()
    run_experiment(n=args.n, rho=args.rho, t=args.t, seed=args.seed, outdir=args.outdir)

if __name__ == "__main__":
    main()

```